CAMPUSNOVA — Phase 0 (Infrastructure)

Goal: Establish a production-grade foundation so Phase 1 (Auth & RBAC) can be implemented c without rework.

Locked decisions (already chosen): - **Monorepo** (single GitHub repo) - **Two React ap** `/apps/admin` and `/apps/school` - Deploy **separate Vercel projects** - **RBAC Option B:** 2 b custom role builder per school **protected permissions** (only System Admin) - **Single Super Admin (root)** with strong security - **Managed SaaS mindset** (Supabase + cloud)

0.1 Product & System Definitions (must exist before code)

1. **Define tenancy model (hard decision)**
2. One "tenant" = one **School**.
3. Decide data isolation strategy:
    - Preferred: shared DB with `school_id` row-level isolation (RLS).
4. **Define environments**
5. `local` → `staging` → `production`.
6. **Define naming conventions**
7. Tables, columns, enums, event names, file paths.
8. **Define MVP scope boundaries for Phase 0**
9. Phase 0 is foundation only (no feature creep).

Deliverable: a short "Phase 0 Definition" doc in repo.

0.2 Repository & Monorepo Setup

1. **Create GitHub repo** (private initially).
2. **Monorepo folder layout** (example):
3. `/apps/admin`
4. `/apps/school`
5. `/packages/ui` (shared UI components)
6. `/packages/config` (shared configs)
7. `/packages/types` (shared TS types)
8. `/packages/api-client` (shared API wrappers)
9. `/supabase` (migrations, policies, seed)
10. `/docs` (architecture + decisions)
11. **Choose monorepo tooling**
12. Recommended: **pnpm workspaces** + **Turborepo** (fast builds, caching).
13. **Set TypeScript project references**
14. Shared `tsconfig.base.json`.
15. **Code quality tools**

16. ESLint
17. Prettier
18. EditorConfig
19. Husky + lint-staged (pre-commit)
20. **Commit conventions**
21. Conventional commits + release tags.

Deliverable: Clean monorepo that installs and builds both apps.

0.3 Frameworks & Core Stack (Phase 0 locked)

Frontend

- **React + TypeScript** for both apps.
- Routing: **React Router** (or your preferred navigation approach, but keep it consistent).
- Data fetching:
- Recommended: **TanStack Query** (caching, retries, request state).
- Forms:
- Recommended: **React Hook Form + Zod**.
- UI:
- Keep minimal in Phase 0 (structure > polish).

Backend / Database

- **Supabase**
- Postgres database
- Auth (Phase 1)
- Storage (later, but scaffold now)
- Edge Functions (only if needed; scaffold optional)

APIs (what "API" means here)

- In a managed Supabase setup, your API surface is:
- Supabase REST (PostgREST)
- Supabase client SDK
- Optional Edge Functions for privileged server-side operations

Deliverable: stack is installed, configured, and standardized.

0.4 Supabase Project Setup

1. Create Supabase projects:
2. `campusnova-staging`
3. `campusnova-prod`
4. Configure auth providers (Phase 0: placeholders only)

5. Create service keys + anon keys
6. Create storage buckets plan (don't overbuild)
7. Enable required Postgres extensions (as needed)

Deliverable: Supabase projects exist and are accessible.

0.5 Database Design Foundations (Phase 0 = structure only)

> You are not building all features now, but you must set a stable foundation for tenanc
> RBAC.

Mandatory tables to define in Phase 0

1. **schools**
2. `id`, `name`, `slug`, `status`, `created_at`
3. **users** (or use Supabase auth users + profile table)
4. Recommended: `profiles` table linked to `auth.users`
5. **school_memberships**
6. user ↔️ school relationship
7. `school_id`, `user_id`, `status`, `created_at`

RBAC foundations (B 2 b)

1. **roles**
2. `id`, `school_id` (nullable for system roles), `name`, `is_system`, `created_at`
3. **permissions**
4. `id`, `key`, `description`, `is_protected`, `created_at`
5. **role_permissions**
6. `role_id`, `permission_id`
7. **user_roles**
8. `user_id`, `role_id`, `school_id`

Audit + safety

1. **audit_logs**
2. actor, action, target, timestamp, metadata

Required DB constraints

- Unique indexes where required (e.g., `schools.slug`, permission `key`).
- Foreign keys and cascading rules.

Deliverable: initial schema with migrations.

0.6 Row Level Security (RLS) Strategy (Phase 0 MUST plan)

1. Decide approach:
2. Every tenant-scoped table includes `school_id`.
3. Define RLS rules pattern:
4. A user can only read/write rows for schools they belong to.
5. Implement baseline RLS policies for:
6. `schools` (read limited)
7. `profiles`
8. `school_memberships`
9. RBAC tables (restricted)

Deliverable: foundational RLS policies (even if Phase 1 refines them).

0.7 Supabase Migrations + Seeding

1. Set up Supabase CLI in repo
2. Standard migration workflow:
3. create migration
4. apply locally
5. push to staging
6. Create seed scripts:
7. One demo school
8. One system admin user (placeholder)
9. A minimal set of permissions
10. A minimal set of roles

Deliverable: repeatable database setup.

0.8 Authentication Plumbing (Phase 0 = scaffolding)

> Do NOT implement full auth flows yet; Phase 1 handles it.

Phase 0 tasks: 1. Wire both apps to Supabase client 2 Create a shared `/auth` module in `packages`: `getSession()` `requireSession()` `signIn()` placeholder `signOut()` 3. Add route guards scaffolding `ProtectedRoute` placeholder components 4. Add “Logged out” UI state placeholder

Deliverable: apps can detect session state (even if login UI comes later).

0.9 Permissions & Authorization Plumbing (Phase 0 = engine shell)

1. Create a shared authorization layer:
2. `hasPermission(user, permissionKey, schoolId)`
3. `hasAnyPermission([...])`
4. `hasRole([...])`
5. Define permission key naming convention:
6. Example: `school.users.read`, `school.users.write`, `rbac.roles.manage`
7. Add protected permissions list strategy:
8. `permissions.is_protected = true`
9. Only Super Admin can create/update protected permissions

Deliverable: permission checks exist (no bypasses).

0.10 API Layer Decision (Client-only vs Edge Functions)

Default (Phase 0): Client SDK

- Use Supabase client for CRUD.

When to require Edge Functions

- Any operation needing:
- elevated privileges
- cross-tenant enforcement
- complex transactions
- secure server-side secrets

Phase 0 tasks: 1. Creat `/supabase/functions` folder (even if empty) 2. Document "When to use Edge Functions"

Deliverable: clear rules so you don't build insecure logic in frontend.

0.11 App Skeletons (Admin + School)

Admin app (`/apps/admin`) must include:

1. App shell layout
2. Route structure skeleton:
3. `/login` (placeholder)
4. `/schools` (placeholder)
5. `/schools/:id` (placeholder)

6. `/permissions` (placeholder)
7. `/roles` (placeholder)
8. Basic navigation + placeholder pages

School app (`/apps/school`) must include:

1. App shell layout
2. Route structure skeleton:
3. `/login` (placeholder)
4. `/dashboard` (placeholder)
5. `/settings` (placeholder)
6. `/users` (placeholder)
7. Basic navigation + placeholder pages

Deliverable: both apps run, routes exist, pages compile.

0.12 Shared Packages

Create shared packages to prevent copy-pa: `/packages/types` - shared domain type `/` 2. `packages/api-client` - Supabase client init - common `/packages/config` - env parsing + validation 4. `/packages/ui` - minimal shared components (buttons/layout)

Deliverable: both apps consume shared packages.

0.13 Environment Variables & Secrets

1. Define env contract (required keys):
2. Supabase URL
3. Supabase anon key
4. environment name
5. Validate env at startup (fail fast)
6. Vercel env setup:
7. separate for admin + school
8. separate for staging + prod
9. Never commit secrets

Deliverable: secure env configuration.

0.14 Logging, Monitoring, and Error Handling (Phase 0 baseline)

1. Frontend error boundary

2. Request error normalization
3. Basic logging strategy
4. Prepare monitoring hooks (Sentry optional)

Deliverable: errors are visible and traceable.

0.15 Security Baselines (Phase 0 minimum)

1. No secrets in client
2. RLS enforced at DB
3. Protected permissions cannot be edited by school admins
4. Audit logging table exists
5. Rate limiting strategy documented (even if not implemented)

Deliverable: foundation is not insecure by default.

0.16 CI/CD and Deployment (Vercel)

1. Create 2 Vercel projects:
2. `campusnova-admin`
3. `campusnova-school`
4. Connect both to same GitHub repo
5. Set root directory per project:
6. Admin: `apps/admin`
7. School: `apps/school`
8. Add build pipeline:
9. lint
10. typecheck
11. build
12. Add preview deployments (PR previews)

Deliverable: staging previews work and prod deploy is controlled.

0.17 Documentation (non-negotiable)

Create `/docs` with: 1. Architecture overview 2. Tenancy + RLS rules 3 RBAC model (B b2) description 4. Environment setup instructions 5. Contribution workflow (branching, PR rules)

Deliverable: anyone can onboard; you can return after weeks and continue.

0.18 Definition of Done (Phase 0)

Phase 0 is DONE only when: - Monorepo installs cleanly and builds both apps - Both apps Supabase project(s) created and connected - Base schema exists via migrations - RLS baseline exi isolation strategy documented + partially enforced) - Shared packages exist and are used - Verce both apps as separate projects - Docs exist for architecture + setup

---

Phase 0 Output Artifacts (what must exist in repo)

- `/apps/admin` runnable
- `/apps/school` runnable
- `/packages/*` shared modules
- `/supabase/migrations/*`
- `/supabase/seed/*`
- `/docs/*`
- CI pipeline config

Next: Phase 1 (Auth & RBAC Implementation)

When Phase 0 is complete, Phase 1 starts immediately with: - Real auth flows - Membersh enforcement - Role assignment UI - Permission checks everywhere - Protected permission admin-o operations